

Topological Feature Learning for Parameter Estimation in Neyman-Scott Processes

First-year progression report

Flavio Gualtieri

August 20, 2026

1 Introduction

The study of spatial point processes finds applications in astronomy, cell biology, epidemiology, etc (Møller and Waagepetersen, 2004). Many natural phenomena – such as galaxy clustering – are modelled using SPPs, so one can hope to leverage an understanding of these processes to obtain novel insights. The parameters of spatial point processes are of particular interest because they are interpretable scientific quantities. [TODO: for example...]

A number of parameter estimation methods have been proposed, typically relying on closed-form expressions for summary statistics, such as the K and L functions (see Section 2.1). Minimum contrast estimation fits parameters by numerically minimizing a discrepancy between the empirical and theoretical curve, most commonly for K or g (Diggle, 2003). Palm likelihood instead builds a composite likelihood from the process’s Palm intensity, and has been developed specifically for Neyman-Scott processes (Tanaka et al., 2008). Fully Bayesian approaches treat the unobserved parent locations as latent variables and sample the posterior over θ by MCMC (Møller and Waagepetersen, 2004). [TODO: add references]

These methods have three limitations we hope to address with our proposed model: ParamNet. Firstly, they rely on the existence of closed-form expressions of summary statistics which may not exist for every process [TODO: for example...]. Secondly, classical parameter estimation methods suffer from hyperparameter sensitivity, specifically the minimum contrast method depends on (r_{max}, q, p) [TODO: how is this different from ParamNet, which also has hyperparameters]. Finally, classical methods become unstable as the process approaches complete spatial randomness (near-Poisson regime). As the overlap index ν grows the clusters merge, so κ and μ cease to be separately identifiable from a single realization [TODO: using lots of undefined notation in this sentence]. Minimum contrast is fitted by numerical optimization of a discrepancy that is close to flat along this trade-off, so the estimates it returns in this regime are sensitive to the starting point and to the choice of r_{max} . We deliberately retain this regime in our design distribution (see Section 3.1) rather than restricting to well-separated clusters.

These limitations motivate the use of persistence-based features which provide a multi-scale, process-agnostic descriptor of the point cloud that does not require a closed-form or a choice of summary statistic.

1.1 Problem setup

Let X be a Neyman-Scott process on $\mathbb{R}^d$ with unknown parameter vector $\theta \in \mathbb{R}^{n_{params}}$. For example, the homogeneous Thomas process has $\theta = (\kappa, \mu, \sigma)$ corresponding to parent intensity, offspring intensity and cluster scale, respectively. We observe a single realization of X on a bounded window $W \subset \mathbb{R}^d$, giving a point cloud $\mathbf{x} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ with $n = |X \cap W|$. The objective is to produce an estimate $\hat{\theta} = \hat{\theta}(\mathbf{x})$ of θ from $\mathbf{x}$ alone. We stress *single realization*: θ is

not recovered from repeated draws of X . This is the setting a practitioner faces: one observed pattern and one chance to estimate the generating parameters. It is also what makes the problem hard in a way no estimator can fix: a fixed θ already induces a distribution over point clouds, and it is this irreducible spread — not any weakness of a particular estimator — that sets the error floor characterized below.

The model is trained on a set of synthetic clouds $\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{train}}$ by backpropagating the RMS loss

$$\mathcal{L}(\hat{\theta}_i, \theta_i) = \sqrt{\sum_{j=1}^{n_{params}} (\hat{\theta}_i^j - \theta_i^j)^2}.$$

The scale of each parameter varies; therefore – if not normalized – the large-scale parameters will dominate this loss. To address this, we train in log-normalized parameter space, mapping $\theta_i \mapsto (\log(\theta_i) - \mu_{\log \theta}) / \sigma_{\log \theta}$, where – abusing notation – $\mu_{\log \theta}, \sigma_{\log \theta}$ denote the mean and standard deviation of the logged parameters over the training set.

We draw one realization per parameter vector. The dispersion of $\hat{\theta}$ across realizations sharing a common θ gives the irreducible error floor against which the model’s performance should be read. We name this quantity $\mathcal{L}_{\text{floor}}$: fix θ , draw R independent realizations, fit $\hat{\theta}^{(r)}$ to each with a fast classical estimator, compute $\mathcal{L}$ between $\{\hat{\theta}^{(r)}\}_{r=1}^R$ and the shared true θ , and average over a grid of θ spanning the design distribution. No estimator that sees only a single realization can average below $\mathcal{L}_{\text{floor}}$ over the same design distribution. [TODO: is this possible for all processes, surely this also depends on a closed form]

2 Background

Vihrs (2022) first proposed the use of deep learning for parameter estimation. They implement an encoder/inference architecture similar to PointNet, but use the L -function as the learned feature and therefore suffer from the same limitations as classical methods (i.e. reliance on a closed-form summary statistic). [REF Yip] employ a similar architecture, but use persistence images as the learned feature. Their model is designed to estimate cosmological parameters rather than the parameters of the underlying point process. Furthermore, they do not systematically compare various topological summaries, nor do they address the calibration of the summaries they use.

The vectorization of persistence diagrams is an active area of research arising from the issue that persistence diagrams – not being a Hilbert space – cannot be directly used as inputs for machine learning models. To mitigate this, Adams et al. (2017) propose persistence images, which vectorise persistence diagrams by applying a Gaussian kernel to each point in the diagram and integrating over a grid. Umeda (2017) proposes Betti curves, which vectorize persistence diagrams by counting the number of topological features that are alive at each scale. [TODO cite other vectorization papers]

The remainder of this section covers the basic notions in spatial point process and persistent homology that the later sections assume.

2.1 Spatial point processes

Definition 2.1: Neyman-Scott process A d -dimensional SPP X on $W \subseteq \mathbb{R}^d$ is equipped with an intensity function $\rho(u) : \mathbb{R}^d \rightarrow \mathbb{R}^+$ such that $\mathbb{E}[N(W)] = \int_W \rho(u) du$, where $N(W) = |W \cap X|$ denotes the number of points of X in W . Let P be a homogeneous Poisson *parent* process with intensity κ . For a Neyman-Scott process with Poisson-distributed offspring we have $\rho_{NS}(u) = \mu \sum_{p \in P} k(u - p)$, where k is a kernel that determines how the *offspring* points are distributed around the parents.

Note Neyman-Scott processes always have the parent and offspring intensities as parameters. Additional parameters are introduced by the choice of kernel. A homogeneous Thomas process is a Neyman-Scott process with an isotropic Gaussian kernel $\mathcal{N}(0, \sigma^2 I)$, and has parameters $\theta_{\text{Thomas}} = (\kappa, \mu, \sigma)$ corresponding to the parent intensity, offspring intensity, and cluster scale, respectively. For stationary and isotropic X , the pair correlation function $g(r) = \rho^{(2)}(r)/\rho^2$ and the K -function $K(r) = \int_{b_r(0)} g(\|h\|) dh$ are the two second-order summaries the classical estimators above are fitted against: $\rho(u)K(r)$ is the expected number of points we expect to see within a distance r of a typical point in X , and g can suggest whether SPPs are attractive ($g > 1$), or repulsive ($g < 1$).

2.2 Persistent homology

Radially averaged statistics – such as the g and K functions – collapse a point cloud to a real-valued curve indexed by scale r , discarding multi-scale connectivity information and void structure, i.e. *how* the density is distributed across the space. Persistent homology retains this information by tracking a family of simplicial complexes on the point cloud as the scale parameter grows. The H_0 persistence summary records the order and scales at which clusters merge; the H_1 counterpart marks voids and gaps in the pattern as they persist across scales. For each $h \in \mathbb{N}$, the h -dimensional persistence summary requires no closed form expression and no assumption about which process family generated it.

Let $\mathbf{x} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be the point cloud of $X \cap W$, and let $\Sigma(\mathbf{x})$ denote the *full simplex* on $\mathbf{x}$, i.e. the collection of all non-empty subsets of $\mathbf{x}$.

Definition 2.2: Filtration A filtration function on $\mathbf{x}$ is a monotone map $f : \Sigma(\mathbf{x}) \rightarrow \mathbb{R}^+$ such that $f(\tau) \leq f(\sigma)$ for all $\tau \subseteq \sigma$. Given $r \geq 0$ we construct $\mathcal{F}_r = \{\sigma \in \Sigma(\mathbf{x}) : f(\sigma) \leq r\}$; monotonicity of f ensures each $\mathcal{F}_r$ is a simplicial complex and that $\mathcal{F}_s \subseteq \mathcal{F}_t$ for $0 \leq s \leq t$.

Example For $u \in \mathbb{R}^d$, let $N_k(u)$ denote its k nearest neighbours in $\mathbf{x}$. The DTM $_k$ filtration function is $f_{\text{DTM}}(\sigma; k) = \max_{u \in \sigma} \{(\frac{1}{k} \sum_{v \in N_k(u)} d(u, v)^2)^{\frac{1}{2}}\}$, extended to higher simplices by a max-over-edges rule (Anai et al., 2020). It is monotone, and is hence a valid filtration function (Anai et al., 2020, Prop. 3.5).

Definition 2.3: Persistence diagram A h -dimensional homology class is *born* at the smallest scale $b \geq 0$ at which it appears in $H_h(\Sigma_b)$, and *dies* at the scale $d > b$ at which it merges into an older class or becomes a boundary; $d - b$ is its *lifetime*. A persistence diagram $D_h = \{(b_i, d_i)\}_i$ is the multiset of dimension- h persistence pairs.

Definition 2.4: Persistence image The persistence image of D_h is constructed by mapping to birth-persistence coordinates $(b, d) \mapsto (b, d - b)$, weighting each point by $w(d - b)$, forming the persistence surface $\rho(z) = \sum_i w(d_i - b_i) \phi(z; (b_i, d_i - b_i))$ – a weighted sum of isotropic Gaussian kernels with bandwidth σ_{im} – and integrating ρ over a 64×64 pixel grid.

3 Progress to date

The work completed so far consists of the simulator and its validation, the ParamNet model and its various architectural ablations, comparison with alternative vectorizations, and benchmarking against existing baselines trained on the same data.

3.1 Synthetic Cloud Generation and experimental design

Training data consists of pairs $(\mathbf{x}_i, \theta_i)$ in which $\theta_i = (\theta_i^1, \dots, \theta_i^{n_{\text{params}}})$ is drawn from a design distribution over parameters space and each $\mathbf{x}_i$ is a single realization of the corresponding process. The naive approach of sampling $(\theta_i^1, \dots, \theta_i^{n_{\text{params}}})$ uniformly and independently from per-parameter intervals is inadequate: point processes can typically be separated into qualitative

regimes of interest, which occupy curved subsets of the parameter grid [explain: what does this mean], so uniform sampling would allocate most of its mass to configurations that are either near-Poisson or degenerate [TODO: why?]. We therefore sample in reparameterized coordinates that carry the same meaning for every family considered: the expected offspring intensity $\lambda = \kappa\mu$, the parent intensity κ , and the dimensionless overlap index $\nu = r\sqrt{\kappa}$, where r denotes the cluster scale of the family in question: $r = \sigma$ for Thomas and $r = \sqrt{\sigma_1^2 + \sigma_2^2}$ for nested Thomas. The overlap index ν measures cluster radius against mean inter-parent spacing, so $\nu \ll 1$ gives well-separated clusters and $\nu \gtrsim 1$ gives clusters that merge. Nested Thomas carries an additional coordinate, the scale ratio $\rho = \sigma_2/\sigma_1 \in (0, 1)$. [TODO: this section is a bit unclear and not obviously general to all NS processes]

We draw $(\log \kappa, \log \lambda, \log \nu)$ uniformly over a grid and invert to recover $\mu = \lambda/\kappa$ and $r = \nu/\sqrt{\kappa}$ for the training labels; log-uniform sampling is used because the parameters span several orders of magnitude. Three constraints are applied to the design grid.

1. *Point budget*: the cost of computing persistence diagrams grows rapidly with the number of points, so we impose $\mathbb{E}[N] = \lambda \in [150, 800]$, fixing the range of $\log \lambda$.
2. *Clusters per window*: we require $\kappa \geq 15$ so that a single realization is informative about κ , and $\kappa \leq 120$ so that individual clusters remain resolvable.
3. *Overlap*: we impose $\nu \in [0.10, 0.90]$, the upper limit deliberately admitting the near-Poisson regime so that the model is still exposed to configurations in which the parameters are weakly identifiable. Parameter draws violating these constraint are rejected and redrawn before any cloud is generated; this filter acts on the design and not on realizations, therefore it does not distort the conditional law of $\mathbf{x}$ given θ .

For a window $W \subset \mathbb{R}^d$, our Neyman-Scott clouds are generated by the following standard construction: [TODO: ref?]:

1. Expand W by an edge buffer b to produce W_{exp} , so that clusters whose parents lie outside W are not truncated.
2. Sample the parent process on W_{exp} : $n \sim \text{Poisson}(\kappa|W_{exp}|)$, with parents placed uniformly on W_{exp} .
3. For each parent p independently, draw an offspring count $n_p \sim \text{Poisson}(\mu)$.
4. For each offspring independently, draw a displacement $\epsilon \sim k$ and place the point at $p + \epsilon$.
5. Discard the parents and retain the offspring lying in W .

The only element that changes from realization to realization is the displacement kernel k . For the Thomas process $k = \mathcal{N}(0, \Sigma^2 I_d)$; for the Matérn cluster process k is uniform on the ball of radius R ; for nested Thomas the parent process at step 2 is itself a Thomas process rather than a homogeneous Poisson process.

3.2 Simulator validation

The validity of the proposed pipeline depends on a diligent cloud-generation process, which we validate against closed-form second-order theory. [TODO: how do we do this for nested thomas?]

On $W = [0, 1]^2$ we compare 3000 realizations at three parameter triples spanning the overlap index: tight-cluster ($\nu = 0.07$), moderate ($\nu = 0.22$), and near-Poisson ($\nu = 1.00$).

For the first moment, $\mathbb{E}[N(W)] = \kappa\mu|W|$. For the second, the canonical identity $\text{Var}[N(W)] = \kappa\mu(1 + \mu)|W|$ holds only in the limit of a window large relative to σ and is badly biased in precisely the near-Poisson regime we wish to stress; the exact finite-window result is

$$\text{Var}[N(W)] = \kappa\mu + \frac{\kappa\mu^2}{4\pi\sigma^2}J(\sigma)^2, \quad J(\sigma) = \int_{-1}^1 (1 - |t|) e^{-t^2/4\sigma^2} dt, \quad (1)$$

for $W = [0, 1]^2$, which collapses to $\kappa\mu(1 + \mu)$ as $\sigma \rightarrow 0$ and is the target we compare against, not the canonical identity.

Case	ν	$\mathbb{E}[N(W)]$		$\text{Var}[N(W)]$	
		theory	empirical	theory	empirical
A	0.07	200.0	200.4 ± 0.6	982.0	972.5 ± 25.1
B	0.22	100.0	100.2 ± 0.4	545.2	544.8 ± 14.1
C	1.00	20.0	20.0 ± 0.1	43.6	46.0 ± 1.2

Table 1: Simulator validation against (1). All deviations are within 2 standard errors.

We additionally verify that the pair correlation function, Ripley’s K , boundary density, and displacement-kernel isotropy all match their theoretical targets.

3.3 Method: ParamNet

Let $\mathbf{x}$ denote a point cloud, v its vectorized persistence summary, and $e \in \mathbb{R}^{E+1}$ the resulting feature vector. ParamNet follows the pipeline [TODO: define E]

$$\mathbf{x} \xrightarrow{\text{Vectorization}} v \xrightarrow{\text{Encoder}} e \xrightarrow{\text{FCNN}} \hat{\theta}. \quad (2)$$

We note that the pipeline shape follows the neural-estimator patterns used in Vihrs (2022) and REF. Our contribution lies in the calibration component of the feature-extraction stage, the application to a more challenging process (Nested Thomas), an explicit evaluation of design choices, and comparison of various TDA-based vectorisations. As mentioned in Section 2 there is substantial literature proposing vectorizations of persistence diagrams – persistence images, Betti curves, landscapes, kernel methods – that trade off stability, resolution, and information loss in different ways, but there has been no formalized discussion on which is preferable in this setting. ParamNet uses the persistence image vectorization [TODO: explain why: because it’s the best one. perhaps another more interesting reason]. We now discuss the key components of the model in turn.

Filtration. Instead of the canonical Rips or Čech filtrations, we use the DTM_k filtration (see Example 2.2). Rips is built from inter-point distances alone, making it sensitive to the sparsest connection between two regions and does not register how densely those regions are occupied. DTM_k instead works with a point’s average distance to its k nearest neighbours, providing a more direct measure of local density. Since the parameters of a point process directly affect the local density of the point cloud, we claim that this choice is natural. We compute a single scale, $k = 5$, noting that error grows monotonically in k on both Thomas and Nested Thomas REF. Persistence features are computed for H_0 only as H_1 seems to contribute nothing measurable to the model’s performance REF.

Calibration. Fix the homology dimension $h \in \{0, 1\}$; we obtain N_{train} persistence diagrams $\{D_i^{(h)}\}_{i=1}^{N_{train}}$, some of which may have extreme outliers relative to the full set. To ensure that in the resulting persistence images each pixel bears the same meaning, the diagrams must be

calibrated to have the same axis ranges. It is not immediately clear that naive approach of calibrating to the maximum deaths and persistences is adequate in this context: if a diagram in the training set has strong outliers, the remaining diagrams will be rescaled in a way that places the majority of their points relatively close the origin. [TODO: figure]

Our solution is to calibrate according to *per-diagram coverage quantiles*. For each training diagram $D_i^{(h)}$, define the smallest birth $b_i^{\min}$, largest birth $b_i^{\max}$, and largest persistence value $p_i^{\max}$ *within a single diagram*. Given a coverage level $q \in (0, 1]$ and a padding factor $\alpha \geq 1$, we set the calibrated birth and persistence ranges to be the q -quantiles of these per-diagram statistics, taken over the training set:

$$\left[Q_{1-q}(\{b_i^{\min}\}_{i=1}^{N_{\text{train}}}), \alpha \cdot Q_q(\{b_i^{\max}\}_{i=1}^{N_{\text{train}}}) \right] \quad \text{and} \quad \left[0, \alpha \cdot Q_q(\{p_i^{\max}\}_{i=1}^{N_{\text{train}}}) \right],$$

where $Q_q(\cdot)$ denotes the empirical q -quantile. These two ranges are shared by every diagram of homology dimension h , for every point cloud in the dataset, so that a given pixel of the resulting persistence image always refers to the same region of birth-persistence space. The calibrated ranges are fit *exclusively* on the training diagrams, then frozen and applied unchanged to validation and test diagrams; fitting them on the full, unsplit population of clouds leaks information about the held-out clouds into the training images. For H_0 , birth values are often identically zero under the chosen filtration, in which case the calibrated birth range collapses to a point, and we fall back to a one-dimensional vectorization. The four calibration parameters – coverage q , padding α , kernel width σ_{pixels} , and raster resolution G – are swept one at a time around default values. Informed by the results of this sweep, ParamNet sets [TODO: add default calibrations]

Encoder and head. For each cloud we compute the H_0 persistence image of the DTM_5 diagram at resolution 64×64 . So for batch size B , the encoding stage is

$$(B, 1, 64, 64) \xrightarrow{\text{Encoder}} (B, E) \xrightarrow{\text{Concatenation}} (B, E + 1)$$

In the encoder stage, the image is passed through a small convolutional stack: three 3×3 convolution blocks with channel widths (32, 64, 128), each followed by a ReLU and 2×2 max-pooling with spatial dropout ($p = 0.2$) between blocks. The resulting feature map is reduced by adaptive average pooling and projected by a linear layer to the embedding dimension E (we use $E = 64$). The concatenation step appends $\log N(\mathbf{x})$, the logged number of points in the cloud, to that embedding. The resulting $(B, E + 1)$ tensor is then passed to the FCNN head $NN_\phi : \mathbb{R}^{E+1} \rightarrow \mathbb{R}^{n_{\text{params}}}$, a small feed-forward network with two hidden layers of width 64 and 32, each followed by a ReLU nonlinearity and dropout ($p = 0.1$), then a final linear layer mapping to the n_{params} outputs. With a single DTM_k scale only one encoder is necessary, and there is no need for an architecture that fuses multiple scales.

Every experiment uses the same optimizer and schedule: AdamW ($\text{lr} = 10^{-3}$, weight decay 3×10^{-4}), batch size 32, up to 500 epochs with early-stopping patience 50 on the validation loss.

3.4 Evaluation protocol

Data are split by parameter vector so that no parameter value appears in both training and evaluation. Beyond the random test split we generate two further evaluation sets: a grid over the design grid, permitting error to be reported as a surface over the recalibrated parameter space used in Section 3.1; and a set drawn one increment outside each range, characterizing degradation under extrapolation. The classical baselines are run on the same clouds and splits rather than on separate data or numbers taken from the literature.

We report error in the log-normalized parameter space of Section 1.1. For a parameter vector with n_{params} components and a test set of size N_{test} , we define the per-parameter loss

$$\mathcal{L}^j = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} (\hat{\theta}_i^j - \tilde{\theta}_i^j)^2 \quad \text{and} \quad \mathcal{L} = \frac{1}{n_{\text{params}}} \sum_j \mathcal{L}^j,$$

where $\tilde{\theta}$ denotes the log-normalized parameter vector and $\hat{\theta}$ the model’s estimate of it. Because the standardization is the one used at training time, a value of $\mathcal{L}^j = 1$ corresponds to the error of a model that has learned nothing beyond the mean of the design distribution. This gives the upper reference point for every figure reported below; the lower reference point is the irreducible error floor of Section 1.1.

3.5 Estimation performance and baseline comparison

The model of Section 3.3 is assembled from choices each of which was measured one axis at a time against the multi-scale, $H_0 + H_1$, coordinate-channel configuration this work started from: dropping to DTM_5 , dropping H_1 , and dropping the coordinate channels each land within one seed-standard-deviation of the arm they were measured against on Thomas. No run configures all three at once, so the figures quoted here and in Table 2 – $\mathcal{L} = 0.126 \pm 0.008$ on Thomas and $\mathcal{L} = 0.196 \pm 0.006$ on nested Thomas – belong to that starting configuration rather than to the model as specified, and the end-to-end confirmation is outstanding.

A dagger (†) marks an arm on which at least one seed failed to converge and collapsed instead to the $\mathcal{L} \approx 0.9$ – 1.0 floor; in these cases the reported mean is dominated by the failed seed(s).

Sanity check: label shuffling. We measure the chance-level floor directly, retraining the default PI model with training labels randomly permuted (inputs unchanged). Both datasets land at $\mathcal{L} \approx 1.0$ (Table 2), giving an empirical floor against which every other result can be read.

What carries the signal. Training on $\log N$ alone (i.e. no topological summary) collapses to the same $\mathcal{L} \approx 0.9$ – 1.0 floor as the label-shuffle control on both datasets, while removing $\log N$ from the full model costs little on nested-Thomas ($0.196 \rightarrow 0.200$) and is merely noisy on Thomas rather than clearly harmful. Together the two arms show the persistence images – not the point count – are carrying the signal, with $\log N$ a minor, possibly non-essential addition.

Method	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
ParamNet (ours)	0.126 ± 0.008	0.196 ± 0.006
vihrs ($L(r) - r$ CNN)	0.115 ± 0.007	0.215 ± 0.004
Minimum contrast, K	0.827 ± 0.646^a	—
Minimum contrast, g	6.167 ± 0.331^b	—
<i>Chance-level reference (label shuffle)</i>	1.011 ± 0.024	0.997 ± 0.018

Table 2: Estimator comparison, both design distributions, $n = 10$ seeds (ParamNet) or $n = 5$ seeds (vihrs, minimum contrast) on the shared test split of Section 3.4. Minimum contrast has no nested-Thomas entry because the process has no closed-form K or g to fit against, here making the comparison itself impossible rather than merely inconvenient. ^a Minimum contrast on K is unstable across seeds rather than uniformly bad: 3/5 seeds land at 0.379–0.399, close to vihrs, while 2/5 land far worse (0.903 and 2.056). ^b Minimum contrast on g is, in contrast, consistently worse than chance on 5/5 seeds – a systematic bias rather than scattered non-convergence. Its contrast exponent $c = 1$ is set by analogy with Diggle (2003)’s K -based rule of thumb rather than tuned; we report the number rather than suppress it, but do not read it as a settled statement about contrast estimation on g in general.

We compare ParamNet against the classical and neural baselines of Section 1: minimum contrast fit to Ripley’s K and, separately, to the pair correlation function g , and `vihrs` – our re-implementation of the neural estimator of Vihrs (2022): a 1-D CNN over the edge-corrected, centred Ripley $L(r) - r$ curve plus $\log N(\mathbf{x})$, i.e. the pipeline of Section 3.3 with the persistence-image feature replaced by a single classical radial summary.

`vihrs` is not merely competitive with ParamNet on Thomas, it is better – and not only in aggregate. Its per-parameter losses ($\mathcal{L}^\kappa = 0.194$, $\mathcal{L}^\mu = 0.123$, $\mathcal{L}^\sigma = 0.029$) are each individually lower than ParamNet’s own (0.204, 0.132, 0.041), so this is not a scalar-averaging artifact: for this single-level process, a hand-built radial summary plus a point count is not a straw-man baseline, and our topological pipeline does not beat it. The picture is different on nested Thomas, where ParamNet’s advantage is concentrated on the density/count targets while `vihrs` keeps a clear edge on both scale parameters. $L(r) - r$ is built to resolve a characteristic radius directly, while a density-based topological feature is not, and it is only on the harder two-level process that this trade-off nets out in the topological pipeline’s favour overall.

Both neural methods dominate minimum contrast in either variant, by roughly an order of magnitude on Thomas and categorically on nested Thomas, where minimum contrast cannot even be attempted. Minimum contrast on g in particular lands *above* the chance-level floor – worse than predicting the design-distribution mean; see the table’s own caption for the caveats around that number before over-reading it. $\mathcal{L}^\kappa$ (parent intensity) is the hardest target for every method on both processes by a wide margin and the cluster scale the easiest, and this ordering holds regardless of which estimator is used – so the difficulty ranking looks like a property of the estimation problem itself rather than an artifact of any one feature set or architecture.

Branches. Each of the following was implemented and run, and none of it is part of the model of Section 3.3; results are collected in Table 3. Error grows monotonically with k within a single scale on both design distributions, consistent with a finer neighbourhood retaining more of the local-density signal the parameters govern while a coarser one averages exactly that structure away; on Thomas the fused arm does not improve on the single best scale, DTM_5 , while on nested Thomas it clearly beats every single scale tried, which is the one place where the model of Section 3.3 is a trade rather than a simplification. Rips fails outright, identically on 10/10 seeds *before* training starts: every H_0 feature is born at $r = 0$, so the calibrated birth range collapses to a point and the calibration step raises rather than silently degrading. H_1 alone lands within noise of the label-shuffle floor while H_0 alone is statistically indistinguishable from the fused $H_0 + H_1$ default; removal of the two CoordConv coordinate channels (Liu et al., 2018) is within seed noise of the full model; and none of the three alternative vectorizations improves on the persistence image, on either process or either encoder path.

Axis	Arm	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Filtration	Rips	— ^a	— ^a
	DTM_5 / DTM_{15}	0.125 ± 0.006 / 0.146 ± 0.006	0.205 ± 0.003 / 0.260 ± 0.016
	$\text{DTM}_{\{5,10,15\}}$ (fused)	0.126 ± 0.008	0.196 ± 0.006
Homology	H_0 only / H_1 only	0.126 ± 0.006 / 0.895 ± 0.019	0.194 ± 0.006 / 0.911 ± 0.013
CoordConv	removed	0.126 ± 0.005	0.194 ± 0.002
Encoder	shared / independent, concat	0.124 ± 0.007 / 0.133 ± 0.005	0.196 ± 0.006 / 0.228 ± 0.029
Vectorization	Persistence image	0.126 ± 0.008	0.196 ± 0.006
	Landscape	0.132 ± 0.008	0.250 ± 0.005
	Silhouette ($p = 1$)	0.165 ± 0.013	0.271 ± 0.012
	Betti curve	0.196 ± 0.014	0.290 ± 0.005
Features	$\log N$ removed	$0.305 \pm 0.392^\dagger$	0.200 ± 0.006
	Topology removed ($\log N$ only)	0.891 ± 0.017	0.946 ± 0.018

Table 3: Branch summary: every axis measured against the multi-scale, H_0+H_1 , CoordConv starting configuration, at the default $q = 0.95$ calibration, $n = 3-10$ seeds per arm depending on the axis. Vectorization rows are the native-encoder path. ^a Rips fails on 10/10 seeds before training starts, for the degenerate-birth reason given in the text.

4 Discussion, limitations and future work

The following are recorded as notes rather than prose.

Limitations.

- **Negative results.** Three alternative vectorizations were implemented and none improved on persistence images. This is evidence about these summaries under this architecture and this design distribution, not a general claim about vectorizations; the same qualification covers multi-scale fusion, H_1 , and coordinate channels, which each fail to earn their place *here*.
- **Where the simplification costs something.** Fused scales beat DTM_5 on nested Thomas (0.196 vs. 0.205), so the single-scale model is a trade on the two-level process and not a free saving.
- **Calibration inherited across summaries.** The per-diagram coverage rule was inherited unchanged by the 1-D summaries for comparability, which may not be optimal for each individually.
- **Single realization; homogeneity.** The estimator sees one realization, and everything here assumes homogeneous, stationary, isotropic processes.
- **The design distribution is a prior.** The design distribution acts as a prior, so the trained model approximates a posterior summary under it, and its ranges must therefore cover the intended application domain – the single most important limitation of any amortized estimator.
- **Computational cost.** Persistence computation scales badly in n ; this is why the point budget of Section 3.1 is capped at 800. *The amortized inference cost per cloud is currently unmeasured.*

Outstanding experiments. *The following results are promised elsewhere in this report but not yet in hand; they are the nearest-term deliverable.*

- Train and evaluate the headline configuration exactly as Section 3.3 specifies it – persistence image, H_0 only, DTM_5 , shared encoder, concatenation with $\log N$, no coordinate channels, $q = 0.95$ – on both design distributions at $n = 10$ seeds, so that the reported $\mathcal{L}$ is the model the report describes.
- **The error floor.** $\mathcal{L}_{\text{floor}}$ does not yet have a value to report: the estimation script was run on a dedicated $50\theta \times 200$ -realization Thomas dataset and every shard completed without error, but every per-cloud loss reduced to NaN rather than a number. One concrete lead: the shard JSONs’ recorded `label_names` include names that do not belong to Thomas’s own (κ, μ, σ) target set, suggesting the per-target loss array is being indexed against the wrong label list. Fix the label-indexing bug and re-run for Thomas; nested Thomas needs either a derived closed form or a different per-realization floor estimator (e.g. `vihrs`, which needs no closed form) before the same route is even available.
- Three evaluation sets defined in Section 3.4 are not yet reported: error as a surface over $(\log \nu, \log \lambda)$ on the grid set – expect degradation as $\nu \rightarrow 1$, and if the model degrades more gracefully than minimum contrast there, that IS the result; degradation on the out-of-range

set; and the Matérn cluster runs for the cross-family comparison – whether one model trained across families beats per-family models is an interesting question in itself, and the common reparameterization of Section 3.1 is what makes it askable.

- [Run the Palm likelihood baseline \(Tanaka et al., 2008\) to completion; it is implemented but has not yet been run.](#)

Planned directions.

- **Near term.** The encoder question: attention over scales, and set-based encoders operating on diagrams directly.
- **Medium term.** Inhomogeneous / non-stationary processes, where classical closed forms are least available and the topological approach has the most to gain – this is arguably where the thesis-scale contribution lies.
- **Medium term.** Real data. Galaxy catalogues if the running example of Section 1 is cosmological.
- **Longer term.** Uncertainty quantification – the model currently emits a point estimate with no interval, which limits its use as a drop-in replacement for likelihood-based methods.

References

- H. Adams et al. Persistence images: a stable vector representation of persistent homology. *Journal of Machine Learning Research*, 18(8):1–35, 2017.
- H. Anai, F. Chazal, M. Glisse, Y. Ike, H. Inakoshi, R. Tinarrage, and Y. Umeda. DTM-based filtrations. In *Topological Data Analysis*, Abel Symposia 15, pages 33–66. Springer, 2020.
- M. Biagetti, A. Cole, and G. Shiu. The persistence of large scale structures. *Journal of Cosmology and Astroparticle Physics*, 2021(04):061, 2021.
- P. J. Diggle. *Statistical Analysis of Spatial Point Patterns*. Arnold, London, 2nd edition, 2003.
- R. Liu et al. An intriguing failing of convolutional neural networks and the CoordConv solution. In *Advances in Neural Information Processing Systems*, 2018.
- J. Møller and R. P. Waagepetersen. *Statistical Inference and Simulation for Spatial Point Processes*. Chapman & Hall/CRC, 2004.
- U. Tanaka, Y. Ogata, and D. Stoyan. Parameter estimation and model selection for Neyman-Scott point processes. *Biometrical Journal*, 50(1):43–57, 2008.
- Y. Umeda. Time series classification via topological data analysis. *Transactions of the Japanese Society for Artificial Intelligence*, 32(3):1–12, 2017.
- N. Vihrs. Using neural networks to estimate parameters in spatial point process models. arXiv preprint, 2022.